Day 4

Authentication Flaws — Breaking Login Logic

Authentication is the process of verifying who you are. Authorization is verifying what you are allowed to do. Both are frequently implemented incorrectly, leading to a wide class of vulnerabilities that let attackers access accounts, data, and functionality they should not be able to reach.

4.1 IDOR — Insecure Direct Object Reference

IDOR is one of the most common and impactful vulnerabilities. It occurs when an application uses a user-controllable value (like an ID number) to access a resource, without checking if the requesting user is allowed to access that specific resource.

Classic Example

```
GET /api/orders/1042  → returns your order
GET /api/orders/1041  → returns someone else's order (IDOR!)
```

The application checks that you are logged in, but does not check that order 1041 belongs to you.

Where to Look for IDOR

- Numeric IDs in URL parameters: /profile?id=123
- Numeric IDs in POST body: {"user_id": 456}
- Filename references: /download?file=invoice_123.pdf
- API endpoints: /api/v1/users/789/data

4.2 JWT Vulnerabilities

JSON Web Tokens are used for stateless authentication. A JWT has three parts separated by dots: Header.Payload.Signature. The payload is only base64 encoded — not encrypted. Anyone can read it.

Decoding a JWT

```
Token: eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyIjoiYWRtaW4ifQ.signature
Decode header:  {"alg": "HS256"}
Decode payload: {"user": "admin", "role": "user"}
```

The Algorithm None Attack

Some JWT libraries accept 'none' as a valid algorithm, meaning no signature is required:

```
Change header to: {"alg": "none"}
Change payload to: {"user": "admin", "role": "admin"}
Remove the signature entirely
Submit the modified token
```

FIX	Always hardcode the expected algorithm in jwt.verify(token, secret, { algorithms: ['HS256'] }). Never trust the algorithm field in the token header.

4.3 Password Reset Flaws

Password reset flows are frequently vulnerable due to weak token generation or flawed logic.

Flaw	Attack
Predictable reset token	Reset tokens based on timestamp or username — guess or brute force
Token not expiring	Old reset links still work — attacker can use a leaked link later
Token in URL	Logged in browser history, proxy logs, referrer headers
Host header injection	Attacker changes Host header so reset link points to attacker's domain
No rate limiting	Brute force OTP codes or short tokens

4.4 Forced Browsing

Applications often hide pages by not linking to them, but never actually protect them with server-side access control. An attacker simply navigates directly to the URL.

```
https://example.com/admin          (try directly)
https://example.com/admin/users
https://example.com/dashboard/export
https://example.com/api/v1/admin/users
```

Tools like Gobuster or Dirb automate this discovery by trying common directory and file names from a wordlist.

4.5 Parameter Tampering

Applications sometimes include role, admin, or price values in requests and trust them without server-side verification.

```
Original request: POST /checkout
{"item_id": 5, "price": 99.99, "qty": 1}
```

```
Tampered request:
{"item_id": 5, "price": 0.01, "qty": 1}
```

If the server uses the price from the request instead of looking it up from the database, the attacker buys items for any price they choose.